

Clash Arena

Projektgruppe: Enzinger Nevio, Ilijevic Alexander

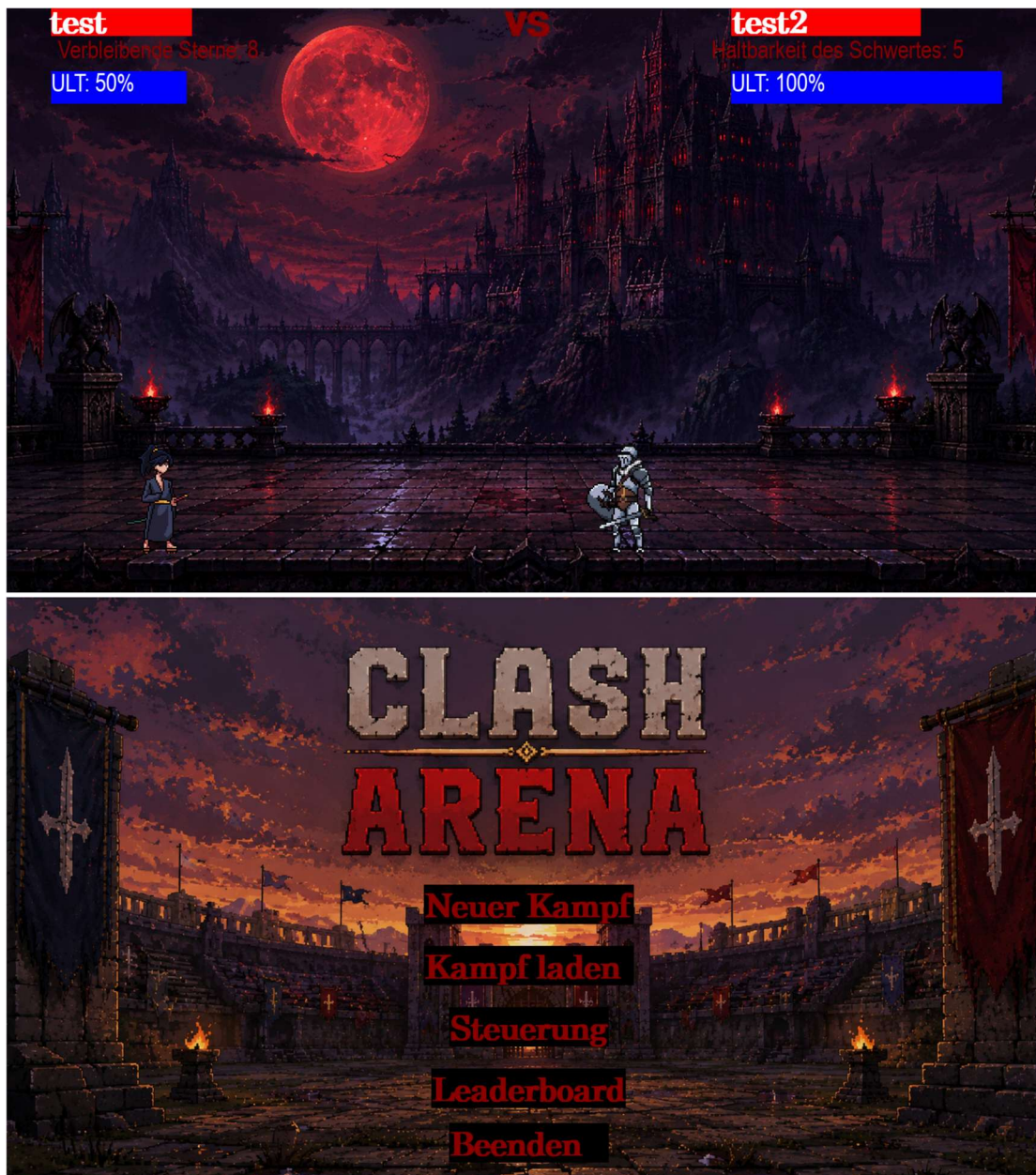
Klasse: 1AHIF

Jahr: 2026

Betreuer: Bechtold David

Kurzbeschreibung:

2 Spieler kämpfen in einer Arena, bis einer der zwei Spieler keine Leben mehr hat.



1 Inhaltsverzeichnis

2 Lastenheft (Kurzbeschreibung, Funktionsumfang, Skizzen).....	3
2.1. Kurzbeschreibung.....	3
2.2. Skizzen	4
2.3. Funktionsumfang	4
.....	17
3 Pflichtenheft	18
3.1 Interner Programmaufbau (Programmlogik)	18
3.2 Ergebnisse, Interpretation (Tests).....	19
4 Anleitung.....	20
5 Bekannte Bugs, Probleme.....	23
6 Erweiterungsmöglichkeiten	24

2 Lastenheft (Kurzbeschreibung, Funktionsumfang, Skizzen)

2.1. Kurzbeschreibung

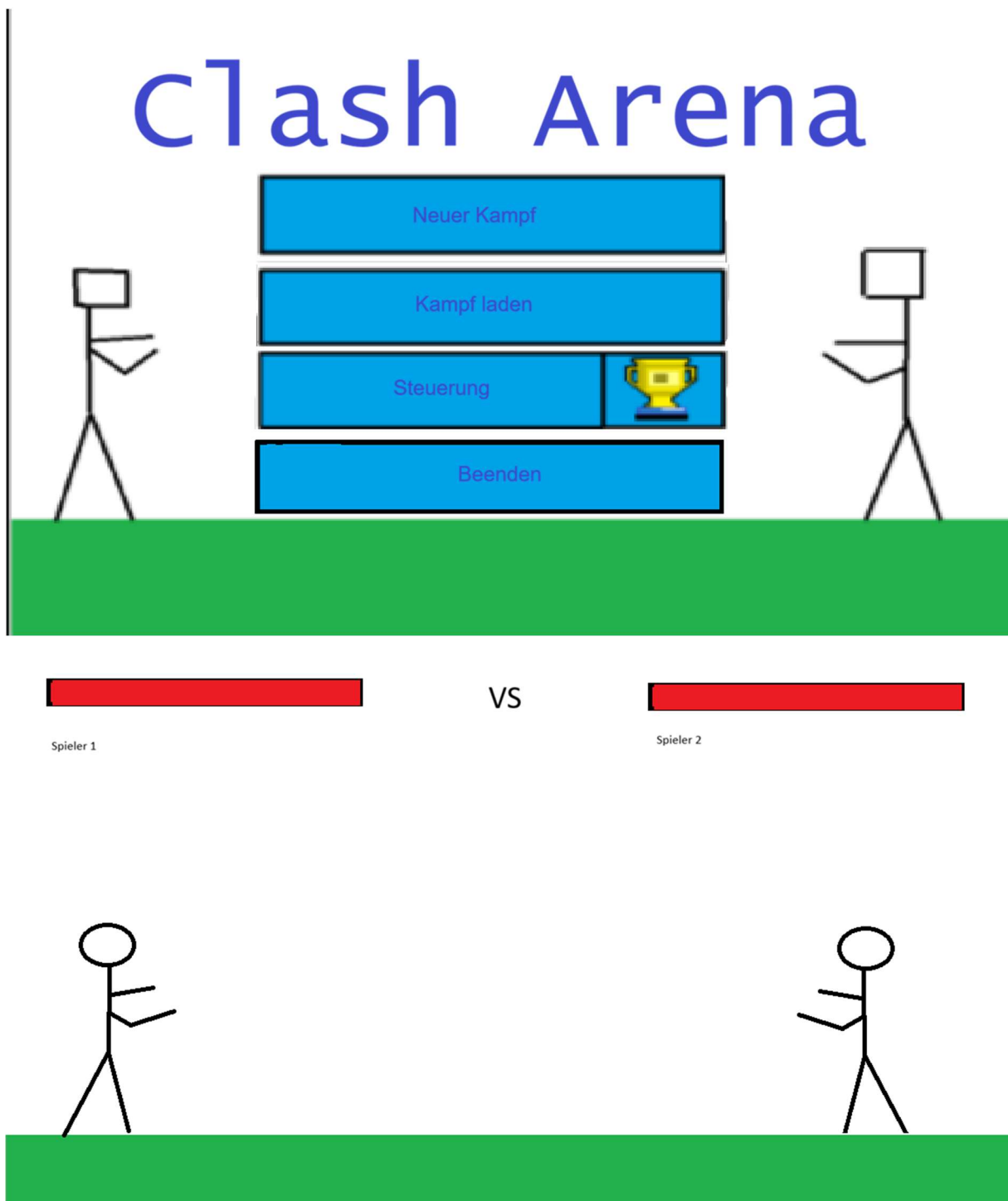
2 Spieler kämpfen in einer Arena, bis einer der zwei Spieler keine Leben mehr hat.

Im Spiel kämpfen zwei Spieler lokal gegeneinander in einer Arena.

Jeder Spieler steuert einen eigenen Charakter mit individuellen Fähigkeiten und versucht, seinen Gegner durch Angriffe zu besiegen.

Das Hauptziel des Spiels ist es, die Trefferpunkte (HP) des Gegners auf 0 zu bringen, bevor man selbst besiegt wird.

2.2. Skizzen



2.3. Funktionsumfang

#KI Anfang

#KI: Claude Code

Prompts: gib mir eine erklärung für die funktionen in screens.py, verändere aber nichts im file selbst, gib mir hier einfach die erklärungen

sort(liste) (Modulfunktion, Zeile 8)

Ein simpler Bubble-Sort. Sortiert eine Liste von Tupeln (name, (level, ...)) absteigend nach element[1][0] (also nach dem Level). Läuft so lange durch die Liste und tauscht benachbarte Elemente, bis ein Durchlauf ohne Tausch passiert (stoppen bleibt True). Wird in highscores genutzt, um die Top-Spieler nach Level zu ordnen.

Screens.__init__

Speichert die übergebenen Grundwerte (FPS, Screen-Objekt, Clock, Breite/Höhe, ScreenVariablen-Objekt SV) als Attribute, damit alle anderen Methoden der Klasse darauf zugreifen können.

error_message(message)

Zeigt einen Fehlertext zentriert auf schwarzem Hintergrund mit einem „Zurück“-Button. Läuft in einer Endlosschleife, bis der Nutzer auf „Zurück“ klickt, dann kehrt die Funktion zurück (kein Returnwert).

choose_character(player)

Zeigt den Charakterausswahl-Bildschirm für einen Spieler (1 oder 2). Lädt und skaliert die drei Auswahlbilder (Ninja, Knight, Vampire), definiert für jedes ein sichtbares Rechteck (*_rect zum Zeichnen) und ein kleineres „Hit-Rechteck“ (*_hit_rect) für die Klickerkennung. Bei Klick auf eines der Hit-Rects wird der entsprechende String ("ninja", "knight", "vampire") zurückgegeben.

enter_name(player) (per KI hinzugefügt, siehe Kommentare im Code)

Zeigt einen Texteingabe-Screen, in dem der Spieler per Tastatur seinen Namen eintippt. BACKSPACE löscht das letzte Zeichen, druckbare Zeichen werden angehängt (max. 15 Zeichen), ENTER bestätigt. Ist der Name leer, wird stattdessen "Spieler {player}" als Standardname zurückgegeben. Bei QUIT wird ebenfalls der Standardname zurückgegeben.

draw_menu()

Reine Zeichenfunktion (kein Loop, kein Event-Handling): rendert die Menüpunkte „Neuer Kampf“, „Kampf laden“, „Steuerung“, „Leaderboard“, „Beenden“ und blättert sie auf den Screen. Wird von menu() in jedem Frame aufgerufen.

todo()

Platzhalter-Screen für noch nicht fertige Features. Zeigt „Noch nicht fertig“ und einen „Zurück zum Menü“-Text. Bei Klick auf den Rückkehr-Text wird "menü" zurückgegeben, bei QUIT "beenden". (Hinweis: der K_ESCAPE-Check ist hier fehlerhaft — event.type == pygame.K_ESCAPE vergleicht den Event-Typ statt event.key, greift also faktisch nie.)

steuerung()

Zeigt den Steuerung-Übersichtsbildschirm mit Buttons für „Spieler 1“, „Spieler 2“ und „Spezialattacken erklärung“. Je nach Klick wird steuerung_anzeigen(1 oder 2) bzw. spezialattacken() aufgerufen, oder bei „Zurück“ wird "menü" zurückgegeben.

laden(screen, SV, clock, FPS)

Lädt einen gespeicherten Spielstand aus saved_game/saved_game.json. Erstellt anhand des gespeicherten Spezialattacken-Typs ("Ninja-Star", "Stab", sonst Vampire) die passenden Charakter-Objekte für Spieler 1 und 2 neu und überträgt die gespeicherten Werte (HP, Position, Spezialattacken-Status, Name). Gibt (player_1, player_2, background) zurück. Schlägt das Laden fehl (z. B. Datei fehlt/korrupt), wird eine Fehlermeldung angezeigt und "menü" zurückgegeben.

steuerung_anzeigen(spieler)

Zeigt die konkreten Tastenbelegungen für Spieler 1 (WASD + E/Q/F) oder Spieler 2 (IJKL + O/U/M) an. Läuft in einer Schleife, bis „Zurück“ geklickt oder das Fenster geschlossen wird. (Auffällig: Hier wird pygame.display.update() statt flip() verwendet, und die Bild-Aktualisierung passiert innerhalb der Event-Schleife statt danach — funktioniert trotzdem, ist aber inkonsistent zum Rest der Datei.)

pausemenu(player_1, player_2, background)

Pausenmenü während eines laufenden Kampfes mit den Optionen „Weiterspielen“, „Steuerung“, „Speichern und Zurück zum Menü“, „Zurück zum Menü“. Beim Speichern werden die aktuellen Spielerdaten (Typ, HP, Position, Spezialattacken-Dict, Name) als JSON in saved_game/saved_game.json geschrieben. Gibt "weiter", "menü" oder "beenden" zurück, je nach Aktion.

play_screen(player_1, player_2, background_filepath)

Die eigentliche Game Loop des Kampfes. Lädt die Hintergrundkarte, zeigt „VS“ an, und in jedem Frame: verarbeitet Events, prüft ESC für Pausenmenü, aktualisiert/zeichnet beide Spieler (update_and_draw), prüft Treffer-Kollisionen für normale und Spezialattacken,

und prüft, ob ein Spieler tot ist (`check_if_dead`). Ist ein Spieler tot, wird der Name des Gewinners zurückgegeben. Sonst läuft die Schleife mit `clock.tick(FPS)` weiter.

highscores(leaderboard)

Zeigt die Top-5-Spieler aus dem übergebenen Leaderboard-Dictionary an. Holt sich per `enumerate` die ersten 5 Einträge, sortiert sie via `sort()` nach Level absteigend und zeigt sie als Liste „Name: Level X“ an. Bei Klick auf „Zurück zum Menü“ kehrt die Funktion zurück.

spezialattacken()

Übersichtsbildschirm mit den drei Charakteren (Ninja, Knight, Vampire), bei deren Klick jeweils `explain_special_attacks(type)` mit dem passenden Spezialattacken-Typ aufgerufen wird. Zeigt zusätzlich den Hinweis, dass Spezialattacken nicht blockbar sind.

explain_special_attacks(type)

Zeigt eine detaillierte Erklärung der Spezialattacke je Charaktertyp ("Stab" = Knight, "Ninja-Star" = Ninja, sonst Vampire): Schadenswert, Abnutzung/maximale Nutzungsanzahl und ein Bild der Waffe/des Effekts. Läuft, bis „Zurück“ geklickt wird.

choose_background()

Analog zu `choose_character`, aber zur Auswahl der Spielkarte. Zeigt dieselben drei Charakterbilder als Auswahl-Icons, gibt aber je nach Klick den Dateipfad der zugehörigen Hintergrundkarte zurück ("assets/play_map2.png", "assets/play_map.png", "assets/play_map3.png").

menu()

Das Hauptmenü. Lädt das Leaderboard aus `leaderboard.json` (falls vorhanden), spielt Hintergrundmusik, zeigt den Menühintergrund und ruft `draw_menu()` zum Zeichnen der Buttons auf. Je nach Klick: „Neuer Kampf“ → gibt "character select" zurück, „Kampf laden“ → "laden", „Steuerung“ → ruft `steuerung()` auf, „Leaderboard“ → lädt das Leaderboard neu und zeigt `highscores()`, „Beenden“ → gibt "beenden" zurück.

#KI Ende

#KI Anfang

KI: Claude Code

Prompt: mache dasselbe jetzt für `ninja.py`

Klasse Star

Repräsentiert einen einzelnen geworfenen Ninja-Stern (sowohl für die normale Spezialattacke als auch für die Ult).

Star.__init__

Speichert Position (xpos, ypos), Geschwindigkeit, Screen-Referenz, Größe und Bildschirmbreite des Sterns.

Star.update_and_draw(liste)

Bewegt den Stern horizontal um speed pro Frame. Verlässt der Stern den Bildschirm (links unter -size oder rechts über screenwidth), wird er aus der übergebenen liste entfernt (pop(0)) und True zurückgegeben (Signal „Stern ist fertig/weg“). Sonst False. Das eigentliche Zeichnen ist hier auskommentiert (Zeile 20) — das Zeichnen passiert tatsächlich über die Sprite-Animationen in Ninja.update_and_draw.

Star.get_rect()

Gibt das Kollisions-Rechteck des Sterns als pygame.Rect zurück, basierend auf Position und Größe.

Klasse Ninja**Ninja.__init__**

Initialisiert den kompletten Zustand eines Ninja-Charakters:

- Grunddaten: Screen, Bildschirmmaße, Spielerposition (xpos/ypos), Größe, HP (100), Schaden, Spielertyp (1 oder 2)
- punch_dict: Zustand der Nahkampfattacke (geworfen, Reichweite, Position)
- special_dict: Zustand der Spezialattacke „Ninja-Star“ (Typ, Liste aktiver Sterne, ob gerade benutzt, verbleibende Sterne = 10, Schaden = 5)
- ult_bar/ult_list/ult_counter: Ultimate-Leiste und die dadurch geworfenen Sterne
- Status-Flags: blocking, jumping, skip_next_punch, pause
- Lädt Sounds (Sprung, Spezialattacke, Punch) und alle Sprite-Animationen (Laufen, Idle, Punch, Block, Sprung, Spezialattacke, Sternflug) jeweils für links/rechts

get_rect()

Gibt das Kollisionsrechteck des Ninja selbst zurück (basierend auf hit_rect-Position und size).

change_x_pos()

Falls der Charakter Spieler 2 ist, wird seine Startposition rechts im Bildschirm gesetzt (screenwidth - size).

change_last_direction()

Setzt die Anfangs-Blickrichtung: Spieler 1 schaut standardmäßig nach rechts, Spieler 2 nach links.

inputs()

Verarbeitet die Tastatureingaben für den jeweiligen Spieler (Spieler 1: WASD+E/Q/S, Spieler 2: IJKL+O/K/U):

- Bewegung (links/rechts laufen), solange nicht geblockt wird und der Charakter nicht am Bildschirmrand ist
- Blocken (F bzw. M)
- Normale Attacke (E bzw. O) — startet den Punch, sofern nicht gerade ein Punch läuft
- Springen (W bzw. I)
- Spezialattacke „Ninja-Stern werfen" (Q bzw. U) — nur möglich, wenn noch Sterne übrig sind und nicht bereits eine Spezialattacke aktiv ist; erzeugt ein neues Star-Objekt in Wurfrichtung und hängt es in `special_dict['list']`
- Ultimate (S bzw. K) — nur bei vollem `ult_bar` (100); wirft ebenfalls einen Stern, aber in `ult_list`, und setzt die Leiste zurück auf 0
- Ist keine Taste gedrückt, wird die Idle-Animation gesetzt

punch()

Solange `punch_dict['thrown']` aktiv ist, wird ein interner counter hochgezählt und die Position der Punch-Hitbox je nach Blickrichtung berechnet. Nach 30 Frames endet der Punch automatisch (counter zurück auf 0, `thrown` auf False).

special_attack()

Solange die Spezialattacke aktiv ist (`special_dict['used']`), wird der erste Stern in der Liste aktualisiert (`update_and_draw`). Verlässt der Stern den Screen, wird `used` zurück auf False gesetzt.

check_punch_collision(enemy)

Prüft, ob der Punch des Gegners (`enemy`) den eigenen `hit_rect` trifft — nur wenn nicht geblockt wird und der Treffer nicht bereits in diesem Wurf gezählt wurde (`skip_next_punch`). Bei Treffer: eigene HP sinken um `enemy.damage`, und die Ult-Leiste des Gegners steigt um 5 (max. 100). `skip_next_punch` verhindert Mehrfachtreffer durch denselben Punch.

draw_healthbar()

Zeichnet HP-Balken (rot, proportional zur HP), Ult-Leiste (blau, proportional zu ult_bar), den Spielernamen, den Ult-Prozentwert als Text und die Anzahl verbleibender Sterne. Position unterscheidet sich je nach player_type (links für Spieler 1, rechts für Spieler 2).

check_if_dead()

Gibt True zurück, wenn hp <= 0, sonst False.

jump()

Steuert die Sprungbewegung: in den ersten 40 Frames steigt der Charakter (ypos -= 6), danach fällt er wieder (ypos += 6). Nach 80 Frames endet der Sprung, jumping wird zurückgesetzt und ypos exakt auf Bodenhöhe fixiert.

check_special_collision(enemy)

Prüft Kollisionen mit Spezialattacken des Gegners, abhängig vom gegnerischen Charaktertyp:

- „Ninja-Star“: Trifft der gegnerische Stern den eigenen hit_rect, sinkt die eigene HP um den Sternschaden, die gegnerische Sternliste wird geleert
- „Stab“ (Knight): Trifft der gegnerische Schwertstoß, sinkt die HP; ignore next verhindert Mehrfachtreffer pro Angriff
- „Blood-Suck“ (Vampire): Trifft der Angriff, sinkt die eigene HP, der Gegner heilt sich um denselben Wert (capped bei 100) und sein blood level steigt um 20

update_and_draw(enemy)

Der zentrale Frame-Update für den Ninja:

1. Ruft inputs() und punch() auf
2. Wählt und zeichnet die passende Animation nach Priorität: Ult-Stern aktiv → Punch → Blocken → Springen → Spezialattacke aktiv → Laufen → Idle
3. Aktualisiert die Punch-Hitbox-Position
4. Falls der Ult-Stern den Gegner getroffen hat (ult_counter > 0), wird alle 20 Frames zusätzlicher Schaden (3 HP) am Gegner verursacht, bis ult_counter nach 240 Frames zurückgesetzt wird
5. Zeichnet die Healthbar
6. Verarbeitet Sprungphysik, falls gerade gesprungen wird
7. Aktualisiert hit_rect auf die aktuelle Position
8. Ruft special_attack() auf und erhöht den frame_counter für die Animationen

#KI Ende

#KI Anfang

KI: Claude Code

Prompt: mache dasselbe für knight.py

Klasse Knight

__init__

Initialisiert den Knight-Charakter:

- Grunddaten: Screen, Bildschirmmaße, Position (xpos/ypos), size (Basisgröße × 2.5), HP (100), Schaden, Spielertyp
- punch_dict: Zustand der Nahkampfattacke (geworfen, Reichweite, Position)
- special_dict: Zustand der Spezialattacke „Stab" (Schwertstoß) — Typ, ob aktiv, Haltbarkeit (8 Nutzungen), Schaden (8), Position, Reichweite, interner Counter
- ult/ult_counter/ult_bar: Ultimate-Zustand (Feuer-Modus)
- Status-Flags: blocking, jumping, ignore/skip_next_punch, pause
- Lädt Sounds (Sprung, Punch, Spezialattacke „Heavy Strike") und alle Sprite-Animationen (Laufen, Idle, Punch, Block, Sprung, Spezialattacke, plus blitz/charge für die Ult-Animation)

change_x_pos()

Setzt für Spieler 2 die Startposition rechts im Bildschirm (screenwidth - size).

change_last_direction()

Setzt die Anfangs-Blickrichtung: Spieler 1 nach rechts, Spieler 2 nach links.

inputs()

Verarbeitet Tastatureingaben für Spieler 1 (WASD+E/Q/S) bzw. Spieler 2 (IJKL+O/U/K):

- Blocken (F bzw. M)
- Bewegung links/rechts (wird wie beim ninja blockiert, wenn self.blocking True ist)
- Normale Attacke (E bzw. O), nur wenn kein Punch läuft
- Springen (W bzw. I)
- Ultimate aktivieren (S bzw. K), nur bei voller Ult-Leiste und nicht blockierend — setzt self.ult = True und leert die Leiste

- Spezialattacke „Stab" (Q bzw. U), nur wenn Haltbarkeit übrig ist, kein Punch und keine Blockade aktiv — verringert die Haltbarkeit um 1
- Standard: Idle-Animation

punch()

Solange `punch_dict['thrown']` aktiv ist, läuft ein Counter hoch und die Punch-Hitbox-Position wird je nach Blickrichtung berechnet. Nach 30 Frames endet der Punch automatisch.

special_attack()

Analog zu `punch()`: solange `special_dict['used']` aktiv ist, läuft ein eigener Counter (`special_dict['counter']`) und die Position des Schwertstoßes wird berechnet. Nach 50 Frames endet die Attacke automatisch (`used = False`).

check_punch_collision(enemy)

Prüft, ob der gegnerische Punch den eigenen `hit_rect` trifft (nur wenn nicht geblockt wird und der Treffer noch nicht gezählt wurde). Bei Treffer: HP sinkt um `enemy.damage`, gegnerische Ult-Leiste steigt um 5 (max. 100).

draw_healthbar()

Zeichnet HP-Balken (rot), Ult-Leiste (blau), Spielernamen und die verbleibende Schwert-Haltbarkeit als Text. Position links für Spieler 1, rechts für Spieler 2.

check_if_dead()

Gibt `True` zurück, wenn `hp <= 0`.

jump()

Identische Sprungphysik wie bei Ninja: steigt 40 Frames lang, fällt danach, endet nach 80 Frames mit fester Bodenposition.

check_special_collision(enemy)

Prüft Treffer durch gegnerische Spezialattacken, je nach gegnerischem Typ:

- „Ninja-Star": Trifft der Stern, sinkt HP, gegnerische Sternliste wird geleert
- „Stab": Trifft der Schwertstoß, sinkt HP; `ignore next` verhindert Mehrfachtreffer pro Stoß
- „Blood-Suck": Trifft der Angriff, sinkt eigene HP, Gegner heilt sich (capped bei 100) und sein `blood level` steigt um 20

update_and_draw(enemy)

Der zentrale Frame-Update, komplexer als bei Ninja wegen des Ult-Feuermodus:

1. Ruft `inputs()` und `punch()` auf
2. Ult-Logik: Ist `self.ult == True`, wird live ein neuer Punch-Sprite („`attack_feuer_mode.png`“) geladen, der Schaden auf 8 erhöht und die Ult-Leiste geleert. In den ersten 60 Frames der Ult wird eine Lade-Animation (`charge`) und ein Blitz-Effekt (`blitz`) gezeigt; nach 180 Frames endet die Ult automatisch.
(Anmerkung: Das Neuladen der Sprite-Objekte bei jedem Frame, in dem `ult == True` ist, ist ineffizient — es lädt das Spritesheet jedes Mal neu von der Festplatte, statt es einmalig zu cachen.)
3. Ist die Ult vorbei (`ignore == False`), wird der normale Punch-Sprite wiederhergestellt und der Schaden auf 2 zurückgesetzt
4. Animationspriorität: (während der ersten 60 Ult-Frames wird nichts gezeichnet, da die Ult-Animation das übernimmt) → Punch → Blocken → Springen → Spezialattacke → Laufen → Idle
5. `hit_rect` wird abhängig von der Blickrichtung neu berechnet (anders positioniert als beim Ninja)
6. Aktualisiert Position von Punch- und Spezialattacke-Hitbox
7. Zeichnet die Healthbar, verarbeitet Sprungphysik
8. Ruft `special_attack()` auf und erhöht `frame_counter`

#KI Ende

#KI Anfang

KI: Claude Code

prompt: mache jetzt eine Erklärung zu den Funktionen von `vampire.py`

Klasse Bat

Repräsentiert die Fledermaus, die der Vampire als Ultimate auf den Gegner hetzt.

Bat.__init__

Speichert Größe, Position, Schaden, Geschwindigkeit und Screen-Referenz. Lädt die Flug-Animation (`bat_spritesheet.png`). `alive` startet auf `False` — die Fledermaus existiert also zunächst inaktiv und wird erst beim Ult-Einsatz „lebendig“ gesetzt.

Bat.update(enemy)

Nur aktiv, wenn `alive == True`. Bewegt die Fledermaus schrittweise (`speed`) auf die Position des Gegners (`enemy.hit_rect`) zu — sowohl horizontal als auch vertikal, jeweils Richtung Gegner. Kollidiert die Fledermaus mit dem Gegner-`hit_rect`, wird dessen HP um `damage` reduziert und die Fledermaus deaktiviert (`alive = False`).

Klasse Vampire

`__init__`

Initialisiert den Vampire-Charakter:

- Grunddaten: Screen, Bildschirmmaße, Position, size (Basisgröße × 2), HP (100), Schaden, Spielertyp
- `punch_dict`: Zustand der Nahkampfatacke
- `special_dict`: Zustand der Spezialattacke „Blood-Suck“ — Typ, ob aktiv, blood level (Start 0), Schaden (10), Position, Reichweite, Counter
- Erstellt ein Bat-Objekt (die Ultimate) mit fixem Schaden 10 und Geschwindigkeit 2
- `ult_bar`: Ultimate-Leiste
- Lädt Sounds (Sprung, Punch) und alle Sprite-Animationen (Laufen, Idle, Punch, Block, Sprung, Spezialattacke) für links/rechts

`get_rect()`

Gibt das Kollisionsrechteck des Vampire zurück, basierend auf `hit_rect`-Position und `size`.

`change_x_pos()`

Setzt für Spieler 2 die Startposition rechts im Bildschirm.

`change_last_direction()`

Setzt die Anfangs-Blickrichtung: Spieler 1 nach rechts, Spieler 2 nach links.

`inputs()`

Verarbeitet Tastatureingaben für Spieler 1 (WASD+E/Q/S) bzw. Spieler 2 (IJKL+O/U/K):

- Blocken (F bzw. M)
- Bewegung links/rechts, gesperrt während des Blockens und am Bildschirmrand
- Normale Attacke (E bzw. O), nur wenn kein Punch läuft
- Springen (W bzw. I)

- Ultimate „Fledermaus losschicken" (S bzw. K) — nur möglich, wenn keine Fledermaus aktiv ist und die Ult-Leiste voll ist; setzt bat.alive = True und positioniert die Fledermaus an der eigenen Position
- Spezialattacke „Blood-Suck" (Q bzw. U) — nur möglich, wenn das blood level unter 100 liegt und keine Attacke bereits läuft
- Standard: Idle-Animation

punch()

Solange punch_dict['thrown'] aktiv ist, läuft ein Counter hoch und die Punch-Hitbox-Position wird je nach Blickrichtung berechnet. Nach 30 Frames endet der Punch automatisch.

special_attack()

Solange special_dict['used'] aktiv ist, läuft ein Counter hoch und die Position des Blood-Suck-Angriffs wird berechnet. Nach 15 Frames endet die Attacke automatisch (kürzer als Knights 50 oder Ninjas Stern-basierte Dauer).

check_punch_collision(enemy)

Prüft, ob der gegnerische Punch trifft (nur ohne Blocken, ohne bereits gezählten Treffer). Bei Treffer: HP sinkt um enemy.damage, gegnerische Ult-Leiste steigt um 5.

draw_healthbar()

Zeichnet HP-Balken (rot), Ult-Leiste (blau), Spielernamen und das aktuelle blood level als Text. Position links für Spieler 1, rechts für Spieler 2. *(Kleiner Tippfehler: bei Spieler 2 steht „BLutlevel" statt „Blutlevel", Zeile 422 — rein kosmetisch, kein Funktionsfehler.)*

check_if_dead()

Gibt True zurück, wenn hp <= 0.

jump()

Identische Sprungphysik wie bei Ninja/Knight: 40 Frames steigen, danach fallen, nach 80 Frames Ende mit fixer Bodenposition.

check_special_collision(enemy)

Prüft Treffer durch gegnerische Spezialattacken — identisch zur Logik in Ninja/Knight:

- „Ninja-Star": HP sinkt, gegnerische Sternliste wird geleert
- „Stab": HP sinkt, ignore next verhindert Mehrfachtreffer
- „Blood-Suck": HP sinkt, Gegner heilt sich (capped bei 100), sein blood level steigt um 20, ignore next verhindert Mehrfachtreffer

update_and_draw(enemy)

Der zentrale Frame-Update:

1. Ruft inputs() und punch() auf
2. Wählt und zeichnet die Animation nach Priorität: Punch → Blocken → Springen → Spezialattacke → Laufen → Idle (kein Sonderfall für die Ult/Fledermaus nötig, da die Fledermaus separat über self.bat.update() gezeichnet wird, nicht über die Charakter-Animation)
3. Aktualisiert Position der Punch- und Spezialattacke-Hitbox
4. Zeichnet die Healthbar
5. Ruft self.bat.update(enemy) auf, um die Fledermaus zu bewegen/zeichnen und Kollisionen zu prüfen
6. Verarbeitet Sprungphysik, falls aktiv
7. Aktualisiert hit_rect, ruft special_attack() auf und erhöht frame_counter

#KI Ende

#KI Anfang

KI: Claude Code

prompt: erstelle ein ablaufdiagramm für das projekt und eine erklärung wie es aufgebaut ist und wie es arbeitet

1. Einstiegspunkt — main.py

Enthält die Top-Level-Spiellogik: initialisiert Pygame/Screen/Clock (SV.init()), erzeugt das zentrale Screens-Objekt und betreibt eine Zustandsmaschine in der main()-Funktion. Eine Variable next_screen (String wie "menü", "character select", "laden", "beenden") bestimmt in jeder Iteration, welcher Screen als Nächstes aufgerufen wird — jede Screen-Methode gibt selbst den Namen des nächsten Zustands zurück. Das ist das Grundprinzip des gesamten Spielflusses.

2. UI-Schicht — screens.py

Die Screens-Klasse bündelt alle Bildschirme (Menü, Charakterauswahl, Steuerung, Leaderboard, Game-Loop selbst usw.) als Methoden. Jede Methode ist eine eigene kleine Eventloop, die blockiert, bis der Nutzer eine Aktion auslöst, und dann zurückkehrt.

3. Charakter-Schicht — ninja.py, knight.py, vampire.py

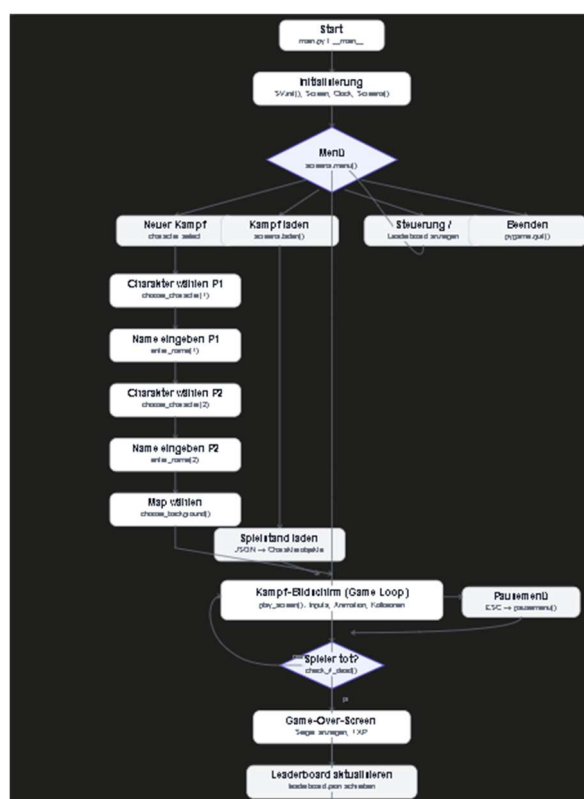
Drei eigenständige Klassen mit identischer Schnittstelle (inputs(), update_and_draw(), check_punch_collision(), check_special_collision(), check_if_dead() ...), aber jeweils eigener Spezialattacke (Ninja-Stern, Schwertstoß, Blutsaugen) und eigener Ult. play_screen() in screens.py ruft diese Methoden für beide Spielerobjekte symmetrisch auf, ohne zu wissen, welcher konkrete Typ dahintersteckt (Duck-Typing).

4. Hilfsschicht — sprite.py, screen_variables.py

Sprite kapselt das Laden/Zeichnen von Spritesheet-Animationen, ScreenVariables hält globale Konstanten (Fonts, Bildschirmmaße, FPS).

Ablauf (siehe Diagramm)

1. Start → Pygame/Screen/Clock initialisieren, Screens-Objekt erzeugen.
2. Menü: vier Wege — Neuer Kampf, Kampf laden, Steuerung/Leaderboard (kehren direkt ins Menü zurück), Beenden.
3. Neuer Kampf: Charakter wählen → Name eingeben (jeweils für Spieler 1 und 2) → Map wählen.
4. Kampf laden: liest gespeicherten Stand aus saved_game.json und baut die passenden Charakterobjekte neu auf.
5. Beide Wege münden im Kampf-Bildschirm (play_screen) — der eigentlichen Game Loop: Inputs verarbeiten, Animationen zeichnen, Kollisionen prüfen. ESC öffnet das Pausemenü (Speichern/Steuerung/Zurück).
6. Stirbt ein Spieler (check_if_dead), folgt der Game-Over-Screen, das Leaderboard wird aktualisiert (leaderboard.json) und es geht zurück ins Menü



3 Pflichtenheft

3.1 Interner Programmaufbau (Programmlogik)

1. Einstiegspunkt — main.py

Enthält die Top-Level-Spiellogik: initialisiert Pygame/Screen/Clock (SV.init()), erzeugt das zentrale Screens-Objekt und betreibt eine Zustandsmaschine in der main()-Funktion. Eine Variable next_screen (String wie "menü", "character select", "laden", "beenden") bestimmt in jeder Iteration, welcher Screen als Nächstes aufgerufen wird — jede Screen-Methode gibt selbst den Namen des nächsten Zustands zurück. Das ist das Grundprinzip des gesamten Spielflusses.

2. UI-Schicht — screens.py

Die Screens-Klasse bündelt alle Bildschirme (Menü, Charakterauswahl, Steuerung, Leaderboard, Game-Loop selbst usw.) als Methoden. Jede Methode ist eine eigene kleine Eventloop, die blockiert, bis der Nutzer eine Aktion auslöst, und dann zurückkehrt.

3. Charakter-Schicht — ninja.py, knight.py, vampire.py

Drei eigenständige Klassen mit identischer Schnittstelle (inputs(), update_and_draw(), check_punch_collision(), check_special_collision(), check_if_dead() ...), aber jeweils eigener Spezialattacke (Ninja-Stern, Schwertstoß, Blutsaugen) und eigener Ult. play_screen() in screens.py ruft diese Methoden für beide Spielerobjekte symmetrisch auf, ohne zu wissen, welcher konkrete Typ dahintersteckt (Duck-Typing).

4. Hilfsschicht — sprite.py, screen_variables.py

Sprite kapselt das Laden/Zeichnen von Spritesheet-Animationen, ScreenVariables hält globale Konstanten (Fonts, Bildschirmmaße, FPS).

Ablauf (siehe Diagramm)

1. Start → Pygame/Screen/Clock initialisieren, Screens-Objekt erzeugen.
2. Menü: vier Wege — Neuer Kampf, Kampf laden, Steuerung/Leaderboard (kehren direkt ins Menü zurück), Beenden.
3. Neuer Kampf: Charakter wählen → Name eingeben (jeweils für Spieler 1 und 2) → Map wählen.
4. Kampf laden: liest gespeicherten Stand aus saved_game.json und baut die passenden Charakterobjekte neu auf.

5. Beide Wege münden im Kampf-Bildschirm (play_screen) — der eigentlichen Game Loop: Inputs verarbeiten, Animationen zeichnen, Kollisionen prüfen. ESC öffnet das Pausemenü (Speichern/Steuerung/Zurück).
6. Stirbt ein Spieler (check_if_dead), folgt der Game-Over-Screen, das Leaderboard wird aktualisiert (leaderboard.json) und es geht zurück ins Menü
7. # KI-Ende

3.2 Ergebnisse, Interpretation (Tests)

Beim Anfang eines neuen Kampfes kann es dazu kommen, dass das Spiel ein wenig ruckelt und man dadurch ein wenig eingeschränkt wird. Die Sprites des Knight-Charakters befinden sich alle am Rand des rects in welches sie gezeichnet werden, wodurch der Knight sich bei Richtungswechseln entweder nach links oder rechts bewegt.

4 Anleitung

#KI Anfang

KI: Claude Code

prompt: erstelle eine Bedienungsanleitung so, dass ein neuer Benutzer mit dem Projekt / spiel klarkommt

1. Spiel starten

1. Stelle sicher, dass Python und die benötigten Pakete (u. a. pygame) installiert sind.
2. Starte das Spiel aus dem src-Ordner heraus mit:
3. Es öffnet sich das Hauptmenü mit Hintergrundmusik.

2. Hauptmenü

Im Menü gibt es fünf Optionen, die du mit der Maus anklickst:

Button	Funktion
Neuer Kampf	Startet einen neuen Kampf: Charakter- und Namensauswahl für beide Spieler, danach Kartenauswahl.
Kampf laden	Lädt den zuletzt gespeicherten Spielstand (aus saved_game.json) und setzt den Kampf an der gespeicherten Stelle fort.
Steuerung	Zeigt die Tastenbelegung für Spieler 1 und 2 sowie eine Erklärung der Spezialattacken.
Leaderboard	Zeigt die Top-5-Spieler nach Level/XP.
Beenden	Schließt das Spiel.

3. Einen neuen Kampf starten

1. Charakter wählen (zuerst Spieler 1, danach Spieler 2): Klicke auf Ninja, Knight oder Vampire.
2. Namen eingeben: Tippe deinen Namen ein (max. 15 Zeichen), bestätige mit Enter. Lässt du das Feld leer, wird automatisch „Spieler 1“ bzw. „Spieler 2“ verwendet.
3. Map wählen: Klicke auf eines der drei Symbole, um die Kampfkarte auszuwählen.
4. Der Kampf beginnt.

4. Steuerung im Kampf

Aktion	Spieler 1	Spieler 2
Laufen links	A	J
Laufen rechts	D	L
Springen	W	I
Normale Attacke	E	O
Spezialattacke	Q	U
Blocken	F	M
Ultimate	S	K
Pausemenü öffnen	ESC	ESC

Hinweis: Spezialattacken können nicht geblockt werden — normale Attacken hingegen schon, solange du blockst.

5. Die Charaktere und ihre Spezialattacken

Charakter	Spezialattacke	Effekt
Ninja	Ninja-Stern (WurfwaFFE)	5 Schaden pro Treffer, max. 10 Mal nutzbar (begrenzte Anzahl Sterne)
Knight	Schwertstoß	8 Schaden pro Treffer, nutzt die Klinge ab, max. 8 Mal nutzbar
Vampire	Blutsaugen	Zieht dem Gegner 10 HP ab und heilt dich selbst um 10 HP, max. 5 Mal nutzbar (begrenzt durch das „Blutlevel“)

Jeder Charakter hat zusätzlich eine Ultimate-Attacke, die erst nutzbar ist, wenn die blaue Ult-Leiste (sichtbar unter dem HP-Balken) zu 100 % gefüllt ist. Die Leiste füllt sich, wenn du Treffer landest. Die Ultimates unterscheiden sich je Charakter (z. B. erhöhter Schaden im „Feuermodus“ beim Knight, eine angreifende Fledermaus beim Vampire).

6. Pausemenü

Drücke im Kampf ESC, um zu pausieren. Du hast dann folgende Optionen:

- Weiterspielen — zurück zum Kampf
- Steuerung — Tastenbelegung erneut ansehen

- Speichern und Zurück zum Menü — sichert den aktuellen Spielstand (HP, Position, Spezialattacken-Zustand, Namen) in saved_game.json und kehrt ins Hauptmenü zurück
- Zurück zum Menü — verlässt den Kampf ohne zu speichern

7. Kampfende

Sobald die HP eines Spielers auf 0 fällt, endet der Kampf automatisch. Der Sieger wird auf einem Game-Over-Bildschirm angezeigt und erhält 10 XP. Sammelt ein Spieler genug XP, steigt sein Level im Leaderboard auf. Mit Klick auf „Zurück zum Menü“ kehrst du ins Hauptmenü zurück.

8. Spielstand laden

Wähle im Menü „Kampf laden“, um den zuletzt gespeicherten Kampf (Charaktere, HP, Position, Spezialattacken-Zustand, Namen, Map) genau dort fortzusetzen, wo du zuletzt gespeichert hast. Konnte der Spielstand nicht geladen werden (z. B. weil noch nie gespeichert wurde), erscheint eine Fehlermeldung und du gelangst zurück ins Menü.

#KI Ende

5 Bekannte Bugs, Probleme

Beim Testen des Spiels konnten wir keine Bugs feststellen, welche wir nicht fixen konnten.

6 Erweiterungsmöglichkeiten

Mehrere Spielbare Charakter
Mehrere Auswählbare Maps
Ninja ultimate Ability rework
Training Modus